

Déploiement DigitalOcean

Ce guide décrit le déploiement d'EcoTrack sur une **DigitalOcean Droplet** avec **Docker Compose**. Pour ce projet, c'est l'option la plus simple et la plus fiable, car le stack inclut plusieurs microservices, PostgreSQL, Redis et Kafka.

Architecture conseillée

- 1 Droplet Ubuntu 22.04 ou 24.04
- 2 vCPU / 4 Go RAM minimum
- 4 vCPU / 8 Go RAM recommandé si tu lances Kafka, PostgreSQL et tous les services sur la même machine
- Les images applicatives sont récupérées depuis Docker Hub via `docker-compose.dockerhub.yml`

Prérequis

- Un compte DigitalOcean
- Un compte Docker Hub
- Un dépôt EcoTrack déjà cloné sur la Droplet
- Les secrets de production prêts : `JWT_SECRET`, mots de passe PostgreSQL et Redis, nom d'utilisateur Docker Hub

1. Créer la Droplet

1. Crée une Droplet Ubuntu.
2. Ajoute ta clé SSH.
3. Connecte-toi en SSH.
4. Mets à jour le système.

```
sudo apt update && sudo apt upgrade -y
```

2. Installer Docker

```
sudo apt install -y ca-certificates curl gnupg git
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) signed-
  by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo \"$VERSION_CODENAME\") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
sudo usermod -aG docker $USER
```

Reconnecte-toi ensuite à la session SSH pour récupérer le groupe `docker`.

3. Récupérer le projet

```
git clone <URL_DU_REPO> ecotrack
cd ecotrack
```

4. Préparer le fichier d'environnement

Copie l'exemple Docker Hub et renseigne les vraies valeurs.

```
cp .env.dockerhub.example .env
nano .env
```

Variables à vérifier en priorité :

- `DOCKERHUB_USERNAME`
- `IMAGE_TAG`
- `JWT_SECRET`
- `POSTGRES_PASSWORD`
- `REDIS_PASSWORD`
- `ALLOWED_ORIGINS`

Si tu utilises un domaine, mets aussi l'URL de production dans `ALLOWED_ORIGINS`.

5. Se connecter à Docker Hub

```
docker login
```

Utilise ton identifiant Docker Hub et un token d'accès si nécessaire.

6. Lancer le stack

```
docker compose -f docker-compose.dockerhub.yml --env-file .env up -d
```

7. Vérifier le déploiement

```
docker compose -f docker-compose.dockerhub.yml --env-file .env ps
docker compose -f docker-compose.dockerhub.yml --env-file .env logs -f --
tail=100
```

Ports exposés par défaut :

- Front citoyen: 3001
- Front admin: 3000
- API Gateway: 3010
- Microservices: 3011 à 3016
- Kafka UI: 8080
- Redis Commander: 8081

8. Ouvrir le firewall DigitalOcean / UFW

Si tu exposes les ports directement, ouvre au minimum ceux utilisés par les applications.

```
sudo ufw allow OpenSSH
sudo ufw allow 3000/tcp
sudo ufw allow 3001/tcp
sudo ufw allow 3010:3016/tcp
sudo ufw allow 8080/tcp
sudo ufw allow 8081/tcp
sudo ufw enable
```

Si tu veux une mise en production plus propre, il vaut mieux mettre un reverse proxy devant les frontends et l'API Gateway, puis laisser PostgreSQL, Redis et Kafka non exposés publiquement.

9. Mise à jour

Quand une nouvelle image Docker Hub est publiée :

```
docker compose -f docker-compose.dockerhub.yml --env-file .env pull
docker compose -f docker-compose.dockerhub.yml --env-file .env up -d
```

10. Dépannage rapide

- Si un service redémarre en boucle, regarde les logs du service concerné.
- Si PostgreSQL n'est pas prêt, vérifie d'abord `POSTGRES_PASSWORD` et les volumes.
- Si Kafka ne démarre pas, vérifie les ports 19092 et 19029.
- Si les frontends appellent la mauvaise API, vérifie `ALLOWED_ORIGINS` et les URLs côté gateway.

Option suivante recommandée

Si tu veux, je peux aussi te préparer une version plus propre pour DigitalOcean avec :

- un reverse proxy Nginx ou Traefik,
- un seul domaine public,
- HTTPS automatique,
- et des ports sensibles non exposés sur Internet.